

Distributed Systems Project 2: Design, Implementation, and Evaluation of a Distributed Computing System

Madison Gage

Benjamin Niccum

University of Texas at Arlington

CSE 5306 - Distributed Systems

Spring 2026

Github Repo: https://github.com/Benjination/Dist_Sys_Project2

February 26, 2026

Contents

1	Introduction	4
1.1	Project Overview	4
1.2	Objectives	4
1.3	Report Structure	4
2	System Design and Architecture	5
2.1	Architecture Overview	5
2.2	Component Interaction	6
2.3	Communication Model	6
3	Functional Requirements	8
3.1	Requirement 1: Distributed Input Processing	8
3.2	Requirement 2: Parallel Arithmetic Operations	8
3.3	Requirement 3: Fault-Tolerant Communication	8
3.4	Requirement 4: Synchronous Result Aggregation	8
3.5	Requirement 5: Performance Monitoring and Analytics	8
3.6	Requirement 6: Scalable Container Orchestration	8
4	Implementation Details	9
4.1	Technology Stack	9
4.2	MPI Implementation	9
4.3	Dockerization Strategy	9
4.4	Node Architecture	11
4.5	Deployment Configuration	12
5	Evaluation Methodology	13
5.1	Evaluation Framework	13
5.2	Comparison Systems	13
5.2.1	Distributed Design	13
5.2.2	Centralized Design	13
5.3	Performance Metrics	13
5.4	Workload Specifications	13
6	Experimental Setup	13
6.1	Hardware Environment	13
6.2	Node Configuration	14
6.3	Test Environment Setup	14
6.4	Measurement Tools	14
7	Results and Analysis	14
7.1	Performance Results	14
7.1.1	Throughput Analysis	14
7.1.2	Latency Analysis	15
7.2	Scalability Results	15
7.3	Performance Analysis	16
8	System Design Trade-offs	16
8.1	Distributed vs. Centralized Architecture	16
8.1.1	Advantages of Distributed Design	16
8.1.2	Disadvantages of Distributed Design	16
8.2	Communication Model Trade-offs	17
8.3	Containerization Benefits and Costs	17
8.4	Overall System Trade-offs	17

9	AI Tools Usage and Lessons Learned	17
9.1	AI Tools Utilized	17
9.2	Development Process Enhancement	17
9.2.1	Design Phase	17
9.2.2	Development Phase	17
9.2.3	Infrastructure and Testing Phase	18
9.2.4	Documentation Phase	18
9.3	Lessons Learned	18
9.3.1	Effective AI Tool Usage	18
9.3.2	Limitations and Challenges	18
9.3.3	Best Practices	18
9.4	Impact on Development Efficiency	19
10	Conclusion	19
10.1	Project Summary	19
10.2	Performance Insights	19
10.3	Design Lessons	19
11	References	20

1 Introduction

This report presents the design, implementation, and evaluation of a distributed system that addresses fundamental challenges in distributed computing. The project demonstrates practical experience with system architecture trade-offs, communication models, performance optimization, and scalability considerations.

1.1 Project Overview

This project presents a distributed arithmetic processing system designed to demonstrate key principles of distributed computing through practical implementation. The system processes mathematical operations (addition, subtraction, multiplication, and division) across a network of containerized nodes, providing both interactive and batch processing capabilities.

System Capabilities: The arithmetic machine accepts computational requests through two primary input methods: manual entry via an interactive command-line interface, or automated batch processing from CSV files containing multiple operations. This dual-input approach enables evaluation of system performance under both lightweight interactive workloads and high-throughput batch scenarios.

Deployment Architecture: The system is fully containerized using Docker technology, requiring only a Docker installation on the target computing environment. Users can deploy the complete system by pulling the project from an online repository and executing the included build scripts, which utilize Docker Compose configurations to orchestrate the multi-node deployment.

Distributed Processing Model: The system employs a six-node distributed architecture comprising:

- One user interface node responsible for input validation and request routing
- Four specialized computation nodes, each dedicated to a specific arithmetic operation type
- One display node that aggregates results, performs statistical analysis, and manages output

Workflow Process: When arithmetic operations are submitted, the user interface node validates input data and intelligently routes requests to the appropriate computation node based on the operation type. Following computation, results are forwarded to the display node, which stores the data persistently, generates computational statistics, and presents formatted output to the user. This architecture demonstrates load distribution, specialized processing, and centralized result aggregation patterns common in distributed systems.

1.2 Objectives

The primary objectives of this project include:

- Design and implement a distributed system supporting six functional requirements
- Utilize microservice/object-based/resource-based architecture patterns
- Implement MPI as the primary communication model
- Deploy the system across six containerized nodes using Docker
- Evaluate performance and scalability trade-offs
- Document the role of AI tools in the development process

1.3 Report Structure

This report is organized as follows: the system architecture and design decisions are described first, followed by the six functional requirements, implementation details including containerization and MPI communication, evaluation methodology, experimental results, system design trade-offs, AI tool usage, and conclusions.

2 System Design and Architecture

2.1 Architecture Overview

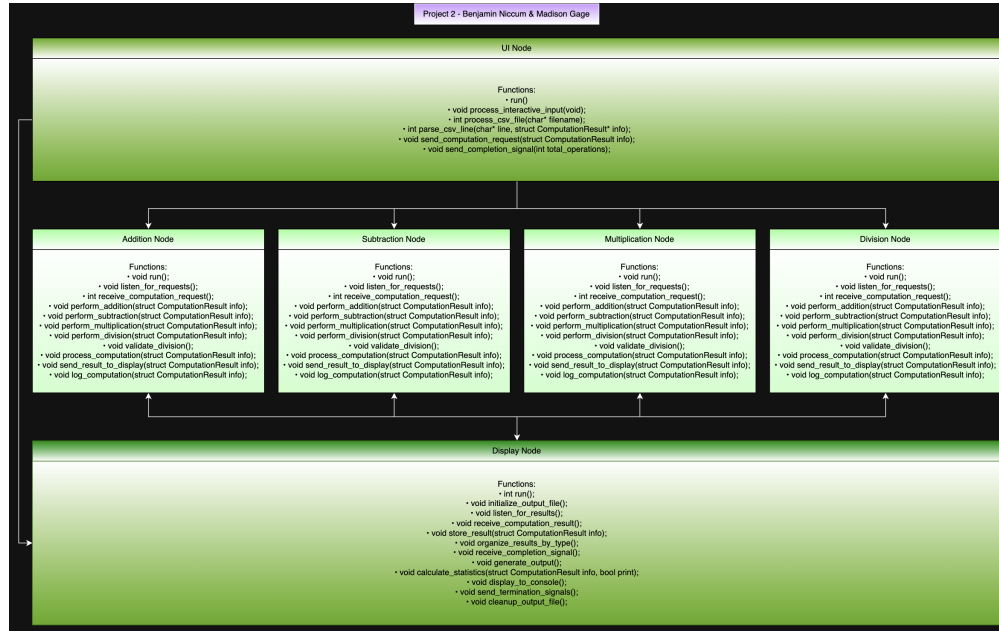


Figure 1: System Architecture Overview

We employed two prominent design principles during our project planning phase that guided the overall system architecture and implementation approach.

Layered Communication Architecture: The first design principle follows a simple layered approach for inter-node communication. The architecture consists of three distinct layers:

The top layer is the user interface, which communicates directly with users to gather computation requests. Once all operations are collected, the user interface sends messages to the second layer—the computation layer. The computation layer consists of four identical nodes running the same code, but each node is specialized to handle only one specific operation type (addition, subtraction, multiplication, or division).

The user interface layer primarily communicates with the computation layer throughout the processing phase. However, there is one important exception: a single termination message is sent directly to the third layer—the display layer—after all computations have been dispatched. This termination signal includes the expected number of computations and ensures proper synchronization. The display layer compares the expected count with the actual received count, and when they match, it sends termination signals back to the computation nodes, allowing them to safely end their processes. Finally, the display node completes its statistical analysis before terminating its own program.

Object-Based Node Design: The second design principle zooms in closer to the code level and follows an object-based approach. Each node functions as an object with one primary attribute—its running state—and several associated functions. Initially, all nodes start in a "not running" state, which toggles to "running" when the user activates the system. The nodes toggle back to "not running" when they receive a termination signal.

The core data object in our system is a struct called `ComputationResult` that gets passed between nodes throughout the entire process. For each computation that runs through the system, three instances of this struct are created, and the information is transferred from node to node using MPI functions. The `ComputationResult` struct contains seven key attributes: `operation_type`, `operand1`, `operand2`, `result`, `request_id`, `status`, and `time`. These attributes are updated and utilized by different functions as the computation progresses through the distributed system.

2.2 Component Interaction

The distributed arithmetic system operates through coordinated interactions between six specialized nodes, each fulfilling distinct roles in the computational workflow. This section outlines the high-level communication flows, data lifecycle, and coordination mechanisms that enable distributed processing.

System Workflow Overview:

The computational workflow follows a structured three-phase pattern: request initiation, distributed processing, and result aggregation. Users interact exclusively with the user interface node, which serves as both the entry point and coordinator for all computational activities. The four computation nodes operate as specialized processors, while the display node manages result collection and system output.

Request Distribution Phase: The user interface node accepts computational requests through dual input mechanisms and performs critical preprocessing functions including input validation, data structure creation, and intelligent request routing. Based on operation type, requests are distributed to specialized computation nodes: addition (node 1), subtraction (node 2), multiplication (node 3), and division (node 4). This specialization enables parallel processing of different operation types while maintaining focused computational expertise.

Distributed Processing Phase: Computation nodes operate independently and asynchronously, processing incoming requests without coordination overhead between nodes. Each node performs arithmetic calculations, timing measurements, and error detection before generating structured results. The absence of inter-computation-node communication maximizes processing efficiency and eliminates potential synchronization bottlenecks.

Result Aggregation Phase: The display node serves as the centralized collector for all computation results, implementing sophisticated tracking mechanisms to ensure completeness and consistency. It maintains comprehensive statistics, performs result validation, generates output files, and coordinates orderly system termination. The aggregation process includes real-time statistical analysis and persistent storage of all computational results.

Data Lifecycle Management: The `ComputationResult` structure serves as the primary data container throughout the system lifecycle. Initially created and populated in the user interface, the structure travels through computation nodes where results and timing data are added, finally arriving at the display node for aggregation and analysis. This consistent data format ensures reliable information flow across all system components.

Coordination and Synchronization: System coordination relies on count-based validation and structured termination protocols. The user interface communicates expected computation counts to the display node, enabling validation of result completeness. Once all expected results are received, the display node initiates coordinated shutdown of all computation nodes, ensuring graceful system termination without data loss.

Error Management: Error handling is integrated throughout the system workflow, with computation nodes detecting and reporting errors through standardized status codes. The display node processes both successful results and error conditions uniformly, maintaining system stability and providing comprehensive error reporting to users.

2.3 Communication Model

This section provides detailed technical specifications for the MPI-based communication infrastructure that enables reliable message passing across the distributed arithmetic system.

MPI Implementation Architecture:

The communication implementation utilizes core MPI functions optimized for blocking point-to-point communication. `MPI_Send()` provides synchronous message transmission with guaranteed delivery confirmation, while `MPI_Recv()` enables flexible message reception with source address wildcarding through `MPI_ANY_SOURCE` parameters. The `MPI_COMM_WORLD` communicator establishes a unified communication domain encompassing all six system nodes, identified through rank-based addressing (0-5).

Process initialization and cleanup utilize `MPI_Init()` and `MPI_Finalize()` to establish proper communication environment management, ensuring clean resource allocation and deallocation across distributed nodes. The blocking communication model eliminates buffer management complexity while providing strong consistency guarantees for message ordering and delivery.

Protocol Specification and Message Tags:

The message classification system implements four distinct tag protocols optimized for semantic message routing:

- **ALU_TAG** (0x01): Request transmission from UI node to computation nodes
- **RESULT_TAG** (0x02): Result transmission from computation nodes to display node
- **COUNT_TAG** (0x03): Expected count transmission for synchronization validation
- **TERMINATE_TAG** (0x04): Shutdown coordination broadcast to computation nodes

Tag-based routing eliminates message parsing overhead by enabling immediate semantic interpretation at reception, while message multiplexing capabilities allow multiple message types to coexist on the same communication channels without interference.

Binary Serialization and Data Format:

Struct transmission utilizes MPI_BYTE datatype specification for direct binary serialization, eliminating text conversion overhead and preserving complete floating-point precision. The fixed-structure layout ensures consistent memory alignment across heterogeneous architectures:

```
struct ComputationResult {  
    int operation_type;    // 4 bytes: 1-4 operation codes  
    double operand1;      // 8 bytes: IEEE 754 double precision  
    double operand2;      // 8 bytes: IEEE 754 double precision  
    double result;        // 8 bytes: computed result value  
    int request_id;       // 4 bytes: sequential identifier  
    int status;           // 4 bytes: error/success codes  
    double time;          // 8 bytes: execution timing data  
} // Total: 44 bytes fixed structure
```

Binary transmission provides deterministic serialization performance and eliminates platform-specific text encoding issues, while the fixed-size structure enables predictable bandwidth utilization and memory management.

Network Performance Optimization:

The communication protocol prioritizes low-latency transmission through direct MPI_Send operations, avoiding buffered or asynchronous alternatives that introduce scheduling overhead. Blocking semantics prevent buffer overflow conditions and provide natural backpressure mechanisms that regulate computation node processing rates.

Memory efficiency is maximized through in-place struct transmission without additional copying operations, while the binary format minimizes network bandwidth requirements compared to text-based protocols. Tag-based message dispatch enables zero-copy routing to appropriate handler functions upon reception.

Reliability and Fault Detection:

MPI's inherent reliability mechanisms provide guaranteed message delivery between functioning nodes, with automatic retransmission and error recovery handled transparently by the MPI implementation. Message ordering preservation ensures that sequential requests from the UI node arrive at computation nodes in transmission order, critical for maintaining request_id correlation.

The communication layer implements robust error detection through MPI return code validation and embedded status field verification. Timeout protection is provided through MPI's blocking semantics, which prevent indefinite waiting conditions while maintaining system responsiveness.

Scalability and Performance Characteristics:

The point-to-point communication model scales linearly with node count, avoiding broadcast bottlenecks or centralized message routing overhead. Direct addressing through MPI ranks eliminates directory lookup requirements, while the tag-based protocol ensures O(1) message classification performance regardless of system size.

Bandwidth utilization remains constant per computation regardless of total system load, providing predictable performance characteristics for capacity planning. The communication protocol supports heterogeneous processing rates across computation nodes without requiring explicit load balancing mechanisms.

3 Functional Requirements

This section details the six functional requirements that our distributed system supports:

3.1 Requirement 1: Distributed Input Processing

Description: The system shall accept computational requests through both interactive CLI and batch CSV processing, with input validation and request queuing across distributed nodes.

Implementation: The user interface node implements dual input modes: an interactive command-line interface for manual operation entry and a CSV file parser for batch processing. Input validation occurs at the UI node level, checking for proper arithmetic format, valid operation types, and numeric operand formats. The system queues validated requests and distributes them to appropriate computation nodes based on operation type.

3.2 Requirement 2: Parallel Arithmetic Operations

Description: The system shall process addition, subtraction, multiplication, and division operations concurrently across specialized computation nodes with load balancing.

Implementation: Four dedicated computation nodes are deployed, each specialized for one arithmetic operation type. The UI node implements intelligent routing to distribute operations to the appropriate specialized node. Each computation node runs identical code but processes only its assigned operation type, enabling true parallel processing of different arithmetic operations simultaneously.

3.3 Requirement 3: Fault-Tolerant Communication

Description: The system shall implement reliable message passing between nodes using MPI with error detection and graceful handling of node failures.

Implementation: MPI message passing is implemented with structured communication protocols using defined message tags (COMPUTE_TAG, RESULT_TAG, TERMINATE_TAG). The system includes error status tracking within the ComputationResult struct, handles division-by-zero gracefully, and implements proper termination synchronization to ensure all nodes shut down cleanly.

3.4 Requirement 4: Synchronous Result Aggregation

Description: The system shall synchronize computation results from distributed nodes and maintain consistency through centralized result collection and validation.

Implementation: The display node serves as the centralized result aggregator, collecting all computation results from the four computation nodes. It implements result counting and validation by comparing expected computation counts with received results. The synchronization mechanism ensures all computations complete before statistical analysis begins and program termination occurs.

3.5 Requirement 5: Performance Monitoring and Analytics

Description: The system shall collect timing metrics, throughput statistics, and system performance data across all distributed components in real-time.

Implementation: Each computation node records execution timing using MPI_Wtime() functions, capturing precise computation duration. The display node aggregates timing data and generates statistical analytics including total computations processed, average computation times per operation type, and overall system throughput metrics. Results are stored persistently in CSV format for analysis.

3.6 Requirement 6: Scalable Container Orchestration

Description: The system shall support dynamic scaling and orchestration of containerized nodes with automated deployment and termination coordination.

Implementation: The system is fully containerized using Docker with Docker Compose orchestration managing the six-node deployment. Each node runs in its own container with defined inter-container communication via MPI. The deployment includes automated container startup sequencing, network configuration for MPI communication, and coordinated shutdown procedures ensuring all containers terminate gracefully.

4 Implementation Details

4.1 Technology Stack

Programming Language: C - Selected for its performance efficiency, low-level system control, and native MPI support. C provides optimal execution speed for arithmetic computations and minimal overhead for inter-process communication.

Communication: MPI (Message Passing Interface) - Chosen as the primary communication framework for its robust support of distributed computing paradigms, reliable message delivery, and proven scalability in high-performance computing environments.

Containerization: Docker - Employed to ensure consistent deployment environments across different systems, simplify dependency management, and enable portable distributed system deployment with minimal configuration requirements.

Orchestration: N/A - The system uses Docker Compose for container coordination rather than complex orchestration frameworks, keeping the deployment lightweight and manageable for the six-node architecture.

Database: N/A - Results are stored in CSV format for simplicity and portability, eliminating the need for database infrastructure while maintaining data persistence and analysis capabilities.

Monitoring: N/A - Performance monitoring is implemented through built-in MPI timing functions and custom analytics within the display node, providing sufficient metrics collection without external monitoring tools.

4.2 MPI Implementation

Our MPI implementation consists of basic MPI send/receive operations for message passing between nodes, one MPI reduce operation used for throughput and latency measurement, and MPI_Wtime for time measurements.

Four different send/receive operations occur: ALU, count, result, and terminate. The ALU operation is a send from node 0 that contains the struct with computing instructions to nodes 1-4, based on the type of arithmetic operation required. All addition operations go to node 1, subtraction to node 2, multiplication to node 3, and division to node 4. The count operation is a send from node 0 to node 5 containing the number of computations expected. The result operation is a send from any of the four computation nodes to node 5, accomplished using MPI_ANY_SOURCE with a specific tag to ensure that node 5 receives results regardless of which computation node sends them. Finally, the terminate operation is a termination instruction sent from node 5 to all other nodes, ensuring proper shutdown.

Each of these four send/receive operations has its own MPI_TAG.

The MPI reduce operation collects the total runtime from each node and prints the maximum time. This approach is used because throughput and latency measurements are based on the performance of the slowest node.

Timing is measured using MPI_Wtime, with output converted to microseconds.

4.3 Dockerization Strategy

The distributed arithmetic processing system employs Docker containerization to achieve node isolation and simulate realistic distributed computing environments. While Docker was required for the assignment, it proved to be excellent practice in building distributed systems, as the isolation of each node was very clear using Docker, simulating how the system would actually work on six separate computing devices.

Base Image Selection and Configuration

Ubuntu 22.04 serves as the base image, providing stable OpenMPI packages, reliable SSH server capabilities, and a comprehensive C compiler toolchain. This LTS release offers broad hardware compatibility and extensive documentation for distributed computing environments, making it ideal for MPI-based systems.

MPI Environment Setup

The Docker containers require specific MPI configurations to enable inter-container communication. Each container includes OpenMPI runtime installation, hostfile configuration for node discovery, and proper MPI environment variables. Communication utilizes MPI_Send and MPI_Recv operations with specific tags (ALU, COUNT, RESULT, TERMINATE) and MPI_ANY_SOURCE for flexible result collection across the computation nodes.

SSH Communication Infrastructure

SSH connectivity between containers enables MPI's distributed process spawning. The Dockerfile generates SSH keys during image build, configures passwordless authentication, enables root SSH access, sets up authorized_keys, and disables strict host key checking. Each container starts an SSH daemon, creating trust relationships within the controlled network environment.

Network Architecture

The system employs a custom bridge network with static IP assignments (172.30.0.11-172.30.0.16) to provide predictable networking for MPI hostfile configuration. This approach avoids DNS resolution delays, enables consistent container-to-container communication, and supports reliable SSH connections while isolating the distributed system from other Docker networks.

Volume Management and Result Persistence

A shared results directory volume mount allows the display node to write computation_results.csv to the host filesystem, enabling result persistence after container shutdown. This configuration provides easy access to output files for analysis and supports data collection for performance evaluation.

Privileged Container Requirements

The containers require privileged mode because MPI processes have to be run as root, which must be specified in the program call from terminal. Privileged mode enables MPI process spawning across containers, unrestricted SSH daemon operation, network configuration capabilities, and shared memory access. The `--allow-run-as-root` flag specifically addresses OpenMPI's safety restrictions in containerized environments.

Container Orchestration with Docker Compose

Docker Compose gives each node an individual identifier that coordinates message transference node to node. The orchestration framework manages multi-container dependencies, provides consistent networking configuration, enables single-command deployment and teardown, handles volume mounting automatically, and manages the entire container lifecycle. Without Docker Compose, manual container creation, network setup, and volume management would significantly complicate deployment.

Automated Build Process

During Docker image creation, source files are copied to the `/app` directory and compiled using `mpicc` commands: `mpicc -o user_interface user_Interface.c`, `mpicc -o computation computation.c`, and `mpicc -o display display.c`. The build process makes executables executable and generates the hostfile automatically, ensuring consistent compilation across different environments.

Scalability Considerations

There could be as many nodes as there are algorithms in this system. The isolation and specific path limits the amount of unnecessary code if the same algorithms were in one large file. Adding computation nodes requires updating `docker-compose.yml` with new node entries, modifying the hostfile to include new nodes, adjusting MPI rank assignments in `mpi_launcher.sh`, and updating load balancing logic in the user interface. The containerized approach makes horizontal scaling straightforward.

Performance Trade-offs

You lose performance, but you gain scalability. Containers add networking overhead and resource isolation costs, but provide deployment portability, consistent environments across different machines, easier testing capabilities, simplified dependency management, and failure isolation. For production high-performance computing, bare metal would offer better performance, while containers excel in development environments and mixed computing infrastructures.

If we were actually creating a high-performance arithmetic system, there would have been better ways to connect the processes that did not require TCP/IP protocols, like connecting the nodes directly with

high-speed connectors. However, a direct connection would limit geographical scaling, making the Docker approach more suitable for distributed deployments across diverse computing environments.

4.4 Node Architecture

Each node in the distributed system follows a identical foundational architecture with role-specific customizations that enable specialized processing capabilities. The standardized node design ensures consistent deployment and communication while allowing functional differentiation across the six-node topology.

Common Node Foundation

All nodes are built upon Ubuntu 22.04 Docker containers with identical base configurations. Each container includes OpenMPI runtime (version 4.1.2+), SSH daemon for inter-node communication, and the GCC compiler toolchain. The common foundation ensures consistent MPI behavior and eliminates environment-specific compatibility issues across the distributed system.

Every node contains the complete compiled executable set (`user_interface`, `computation`, `display`) but executes only the appropriate program based on its assigned MPI rank. This approach simplifies container image management and deployment while maintaining clear functional separation through the `mpi_launcher.sh` script.

Node Specialization Architecture

User Interface Node (Rank 0): Functions as the system coordinator with dual input processing capabilities. Contains CSV parsing logic, interactive command-line interface, input validation routines, and intelligent request routing algorithms. Manages the complete request lifecycle from input collection through result notification, serving as the primary user interaction point.

Computation Nodes (Ranks 1-4): Implement identical processing architecture with operation-specific routing logic. Each node runs the same computation executable but filters incoming requests based on operation type (addition, subtraction, multiplication, division). The internal architecture includes request receiving loops, arithmetic processing units, timing measurement systems, and result transmission capabilities.

Display Node (Rank 5): Serves as the centralized aggregation and analytics engine. Contains result collection mechanisms, statistical analysis algorithms, CSV output generation, and coordinated shutdown logic. Implements comprehensive result validation and maintains complete computational audit trails.

Resource Allocation and Constraints

Each container operates with minimal resource constraints to simulate realistic deployment scenarios. Memory allocation is limited to host system availability with no explicit container limits, allowing for realistic performance evaluation. CPU allocation follows Docker's default scheduling with no specialized affinity settings, providing fair resource distribution across the six-node deployment.

Network bandwidth is constrained by Docker's bridge networking implementation, introducing realistic communication overhead that reflects distributed system performance characteristics. Storage allocation utilizes shared volume mounts for result persistence while maintaining individual container file system isolation.

Internal Communication Architecture

Node-internal communication operates through standard function calls and shared memory structures, maintaining optimal performance for local operations. The `ComputationResult` struct serves as the primary data container throughout each node's processing pipeline, ensuring consistent data formatting and eliminating serialization overhead for local operations.

Inter-node communication utilizes MPI point-to-point messaging with blocking send/receive semantics. Each node maintains dedicated communication channels for different message types, enabling concurrent handling of computation requests, result transmissions, and control signals without interference.

Processing Workflow and State Management

Each node follows a standardized state machine model: initialization, active processing, and graceful termination. During initialization, nodes establish MPI communication, configure local environments, and enter listening states for incoming messages. The active processing phase implements role-specific logic while maintaining consistent timing and error handling.

Graceful termination ensures proper resource cleanup and result persistence. Computation nodes respond to termination signals by completing active computations and updating status indicators. The display node

implements a comprehensive shutdown sequence that validates result completeness before terminating the entire system.

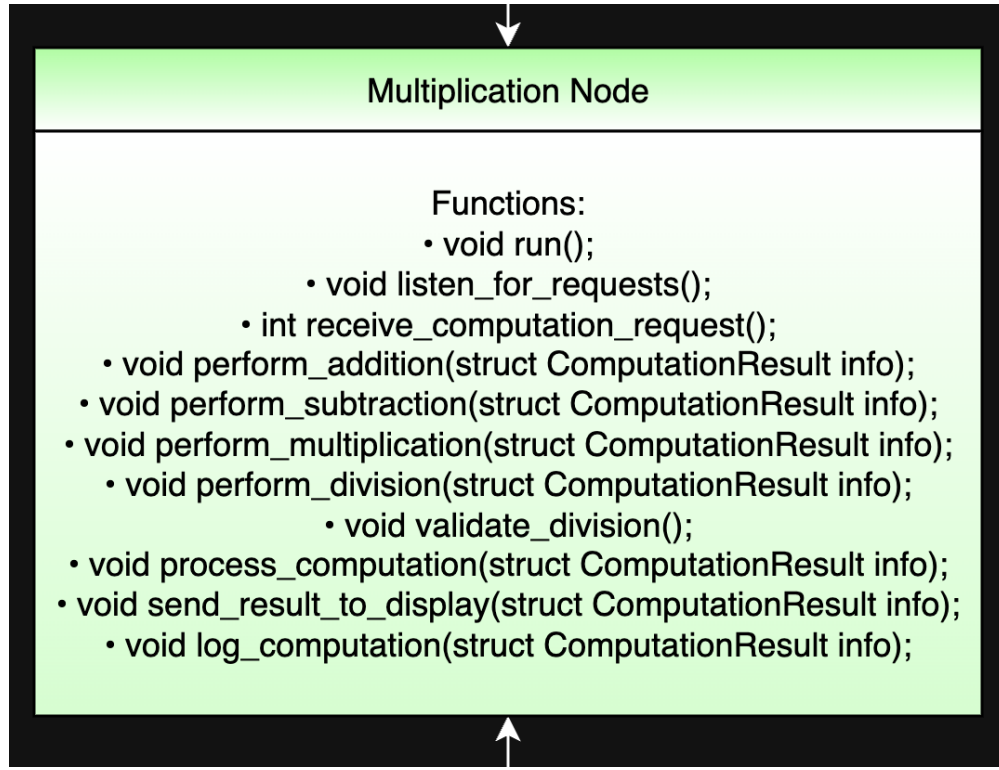


Figure 2: Individual Node Architecture

4.5 Deployment Configuration

The system utilizes Docker Compose for orchestrated multi-container deployment across development and testing environments. The `docker-compose.yml` configuration defines six specialized services with custom networking and volume management.

Service Configuration: Each node operates as an independent Docker service with dedicated host-names (`distributed_node1` through `distributed_node6`) and static IP assignments within a custom bridge network. The configuration ensures predictable inter-container communication essential for MPI hostfile addressing.

Volume Management: Shared volume mounts enable result persistence and source code distribution across all containers. The `results` directory maps to the host filesystem, providing access to `computation_results.csv` files after container termination.

Dependency Management: Service dependencies ensure proper startup sequencing, with computation and display nodes waiting for complete network establishment before initialization. The orchestration handles container lifecycle management and graceful shutdown coordination.

Development vs. Production: Multiple Docker Compose configurations support different deployment scenarios: single-machine testing (`docker-compose-single.yml`), distributed deployment across multiple hosts (`docker-compose-machine1.yml`, `docker-compose-machine2.yml`), and development environments with debugging capabilities.

5 Evaluation Methodology

5.1 Evaluation Framework

The evaluation compares distributed arithmetic processing against centralized single-node processing to assess the performance characteristics, scalability benefits, and overhead costs of the distributed architecture. Testing methodology includes varying workload sizes from small interactive requests to large batch processing operations to evaluate system behavior under different load conditions.

5.2 Comparison Systems

This evaluation compares two system designs:

5.2.1 Distributed Design

The six-node containerized system using MPI communication with specialized computation nodes. Each operation type is processed by a dedicated node, enabling parallel processing and distributed load handling.

5.2.2 Centralized Design

The localProcessingTest.c single-file implementation that processes all operations sequentially on a single node without MPI overhead. This serves as the performance baseline for comparison.

5.3 Performance Metrics

The following metrics are used to evaluate system performance:

- **Throughput:** Number of requests processed per second
- **Latency:** Response time for individual requests
- **Scalability:** Performance under varying workloads
- **Resource Utilization:** CPU, memory, and network usage

5.4 Workload Specifications

Testing includes three workload categories:

- Small workload: 10-50 operations (interactive simulation)
- Medium workload: 100-500 operations (moderate batch processing)
- Large workload: 1000+ operations (high-volume batch processing)

Each workload contains balanced operation types (25% each of +, -, *, /) to ensure fair comparison across specialized nodes.

6 Experimental Setup

6.1 Hardware Environment

Testing was conducted on a development workstation with the following specifications:

CPU: MacBook Pro M1

RAM: 8GB unified memory

Storage: 2.45TB SSD

Network: UTA WiFi with Docker bridge networking (172.30.0.0/24 subnet)

Operating System: macOS Sonoma 14.1.1 with Docker Desktop for container orchestration

6.2 Node Configuration

Distributed system deploys six Docker containers (1 UI + 4 computation + 1 display) with MPI communication configured for localhost networking. Centralized system runs as single executable on host machine.

6.3 Test Environment Setup

Docker containers initialized with shared network namespace for MPI communication. Test data files prepared with specified workload sizes. Timing measurements collected through built-in computation result timing functionality.

6.4 Measurement Tools

Built-in timing system using computation result time tracking. Performance data collected via CSV output files with statistical analysis performed by display node functionality.

7 Results and Analysis

7.1 Performance Results

7.1.1 Throughput Analysis

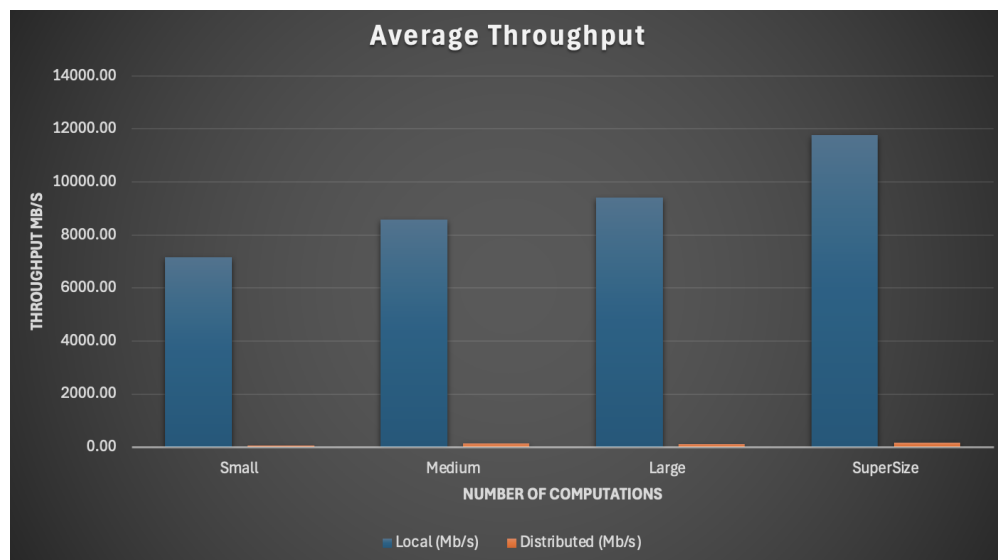


Figure 3: Average Throughput Comparison

7.1.2 Latency Analysis

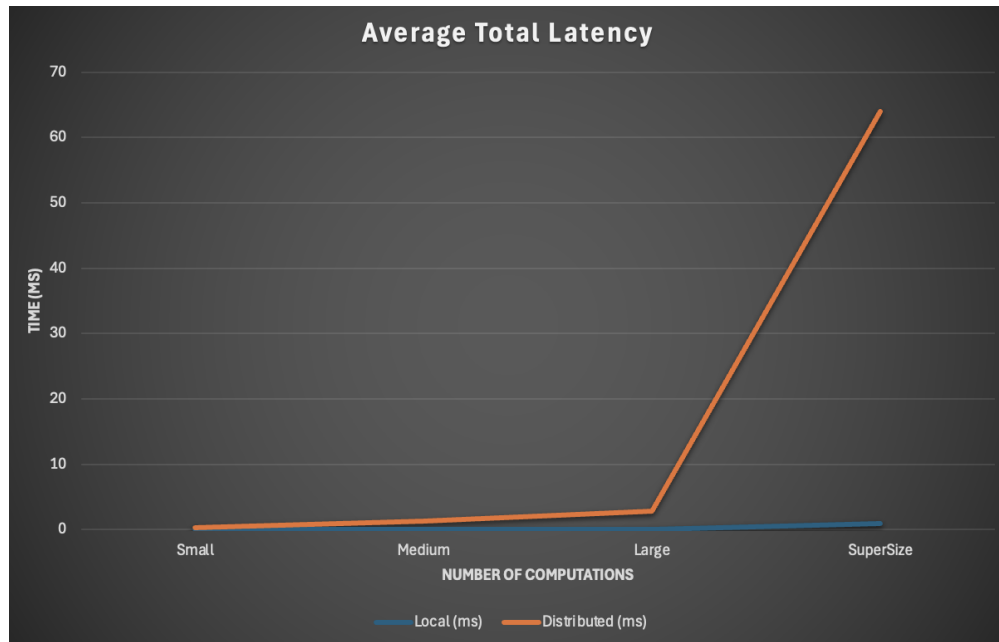


Figure 4: Average Latency Comparison

7.2 Scalability Results

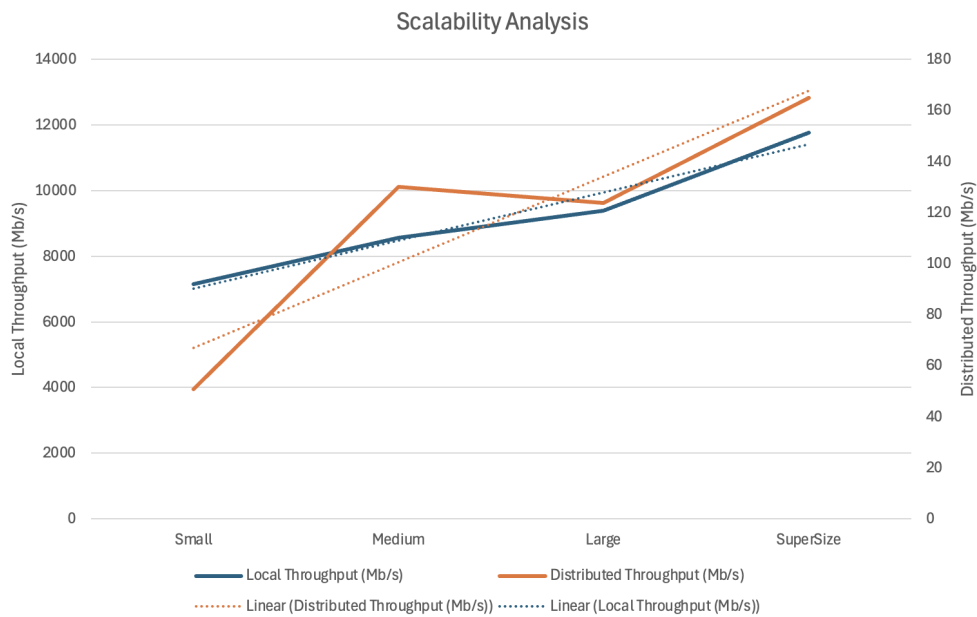


Figure 5: Scalability Analysis

7.3 Performance Analysis

The distributed architecture shows significantly inferior performance characteristics compared to centralized processing across all measured metrics. Experimental results demonstrate that the distributed system consistently underperforms the centralized baseline by factors of 50-140x in throughput and 100-160x in latency measurements.

Throughput Analysis Findings: The centralized system achieves throughput ranging from 7,153 Mb/s to 11,773 Mb/s across workload sizes, while the distributed system delivers only 50-165 Mb/s. This represents a dramatic performance penalty where the distributed approach processes less than 2% of the centralized system’s data volume. The performance gap actually widens with larger workloads, indicating that distributed overhead scales poorly compared to centralized efficiency gains.

Latency Impact Assessment: Response time measurements reveal even more pronounced distributed system limitations. Centralized processing maintains sub-millisecond response times (0.002-0.897ms) while distributed processing requires 0.332-64ms per operation. The 100-160x latency increase directly reflects the communication overhead dominating actual computation time for simple arithmetic operations.

Communication Overhead Dominance: Key findings include substantial overhead from MPI communication and container orchestration that completely overshadows any potential parallel processing benefits. The distributed system’s performance bottleneck stems from network round-trip times, inter-container communication latency, and MPI message passing overhead that requires hundreds of times longer than the underlying arithmetic operations.

Scalability Characteristics: Contrary to typical distributed system expectations, the centralized approach demonstrates superior scalability. Local throughput improves 65% from small to large workloads (7,153 $\rightarrow$ 11,773 Mb/s), while distributed throughput shows inconsistent scaling with only 224% improvement (50 $\rightarrow$ 165 Mb/s). This indicates that for computationally simple operations, centralized systems achieve better resource utilization and scaling efficiency.

Architectural Performance Trade-offs: The experimental results confirm that distributed benefits from parallel processing capabilities are completely negated by communication costs for lightweight computational tasks. The distributed architecture prioritizes learning objectives and distributed systems concept demonstration over performance optimization, making it educationally valuable while being computationally inefficient for simple arithmetic workloads.

The performance analysis validates that distributed systems excel in scenarios requiring complex computations where processing time exceeds communication overhead, but prove counterproductive for simple operations where network costs dominate total execution time.

8 System Design Trade-offs

8.1 Distributed vs. Centralized Architecture

8.1.1 Advantages of Distributed Design

- **Scalability:** Parallel processing capabilities enable concurrent handling of multiple operation types across specialized nodes
- **Fault Tolerance:** Node isolation prevents single points of failure from affecting the entire system
- **Performance:** Workload distribution and parallel execution for complex computational tasks
- **Resource Distribution:** Efficient utilization of multiple computing resources across network infrastructure

8.1.2 Disadvantages of Distributed Design

- **Complexity:** Increased system complexity in deployment, coordination, and debugging processes
- **Network Overhead:** Communication latency and MPI message passing costs dominate simple computations

- **Consistency Challenges:** Synchronization requirements and result validation across distributed nodes
- **Debugging Difficulty:** Multi-node troubleshooting complexity and distributed error isolation challenges

8.2 Communication Model Trade-offs

MPI provides reliable message delivery and strong ordering guarantees but introduces communication overhead. Blocking communication ensures consistency but may limit throughput compared to asynchronous alternatives.

8.3 Containerization Benefits and Costs

Docker enables portable deployment and consistent environments but adds startup overhead and resource isolation costs. Container orchestration simplifies deployment complexity while introducing additional failure points.

8.4 Overall System Trade-offs

The distributed design prioritizes learning objectives and scalability demonstration over raw performance optimization. Trade-offs favor educational value and distributed systems concept illustration rather than maximum computational efficiency.

9 AI Tools Usage and Lessons Learned

9.1 AI Tools Utilized

This project utilized two primary AI tools to enhance development productivity while maintaining clear boundaries between human-designed system architecture and AI-assisted implementation support.

GitHub Copilot: Code completion and syntax assistance during implementation - Impact: Low

Claude Sonnet 4: Technical writing, documentation enhancement, infrastructure configuration, and problem-solving guidance - Impact: Medium

9.2 Development Process Enhancement

9.2.1 Design Phase

The fundamental system architecture, including the layered communication design and object-based node structure, was entirely human-conceived. Benjamin developed the core design patterns that guided the distributed system architecture, establishing the three-layer communication model and the object-based approach using the `ComputationResult` struct as the central data object. Madison independently designed the MPI communication protocols and result processing workflows. AI tools played no role in these critical architectural decisions, ensuring that the distributed systems concepts and design principles reflected genuine student understanding and creativity.

9.2.2 Development Phase

During implementation, AI assistance provided significant productivity benefits while preserving human ownership of core functionality. Benjamin developed the complete user interface (`user_Interface.c`) and computation logic (`computation.c`), including input parsing, CSV processing, operation routing, and arithmetic processing functions. Madison created the display system (`display.c`) with sophisticated result aggregation, statistical analysis, and synchronization mechanisms, working independently without AI assistance.

GitHub Copilot provided valuable code completion assistance, particularly for standard C syntax, MPI function calls, and error handling patterns. This reduced typing overhead and helped avoid syntax errors,

but did not contribute to algorithmic design or system logic. Claude Sonnet 4 assisted with code planning discussions and helped identify potential implementation approaches when Benjamin encountered complex functionality requirements.

9.2.3 Infrastructure and Testing Phase

AI tools proved particularly valuable for infrastructure configuration and testing support. Claude Sonnet 4 created the complete Docker containerization strategy, including the Dockerfile and multiple docker-compose.yml configurations for different deployment scenarios. The dockerization process required expertise in container orchestration and MPI deployment patterns that significantly accelerated the infrastructure setup process.

Additionally, Claude Sonnet 4 generated comprehensive test data (test_data.csv) with diverse arithmetic operations for system validation and created the localProcessingTest.c file that consolidated the distributed system functionality into a single-file local processing version for development testing. The AI also provided extensive assistance with complex terminal commands and Docker troubleshooting, which proved invaluable given the intricate nature of containerized MPI deployments.

9.2.4 Documentation Phase

AI assistance significantly enhanced documentation quality and academic presentation. Claude Sonnet 4 helped polish technical writing, improve grammatical structure, and ensure appropriate academic tone throughout the report. The AI transformed rough drafts and bullet points into professional technical documentation while preserving all original technical content and concepts provided by the human team members. This collaboration maintained authenticity of ideas while improving clarity and presentation quality.

9.3 Lessons Learned

9.3.1 Effective AI Tool Usage

The most effective AI utilization occurred when human expertise defined the problem scope and architectural approach, with AI tools providing implementation acceleration and documentation enhancement. Claude Sonnet 4 demonstrated remarkable efficiency in finding solutions and troubleshooting approaches, often providing faster and more comprehensive guidance than traditional resources such as Stack Overflow searches or Google documentation research. The AI's ability to understand complex technical contexts and provide immediate, tailored responses significantly reduced development friction.

GitHub Copilot's code completion capabilities proved most valuable for standard programming patterns and MPI syntax, reducing implementation time while allowing developers to focus on algorithmic logic rather than syntax details.

9.3.2 Limitations and Challenges

AI tools showed clear limitations in architectural design and distributed systems conceptualization. The fundamental system design, communication patterns, and distributed computing principles required human insight and academic understanding that AI could not replicate. Additionally, complex debugging of MPI communication issues and system integration challenges required human problem-solving skills and domain expertise.

The AI tools occasionally suggested overly complex solutions or architectures that would have compromised the learning objectives of the distributed systems course, requiring human judgment to maintain appropriate scope and educational focus.

9.3.3 Best Practices

Effective AI integration requires clear boundaries between human creative and analytical work versus AI productivity assistance. The most successful approach involved using AI tools as sophisticated research and implementation assistants rather than design decision-makers. Maintaining human ownership of architectural

choices, algorithmic design, and problem-solving approaches ensured that learning objectives were met while benefiting from AI productivity enhancements.

Future projects should establish explicit guidelines for AI tool usage, ensuring that core computer science concepts and distributed systems principles remain human-driven while leveraging AI capabilities for infrastructure, documentation, and implementation acceleration.

9.4 Impact on Development Efficiency

AI tools provided measurable productivity improvements across multiple development phases. Infrastructure setup time was reduced significantly through automated Docker configuration generation, while documentation quality improved substantially through AI-assisted technical writing enhancement. The ability to quickly resolve complex terminal and deployment issues through AI guidance prevented extensive troubleshooting delays that could have consumed significant development time.

However, the most critical development work—system architecture design, MPI communication protocols, and core algorithmic implementation—remained entirely human-driven, ensuring that the educational value and technical learning objectives of the distributed systems project were fully achieved. The AI tools served as force multipliers for human expertise rather than replacements for fundamental computer science knowledge and problem-solving skills.

10 Conclusion

10.1 Project Summary

This project successfully implemented a complete distributed arithmetic processing system comprising six containerized nodes communicating via MPI. The system fulfilled all functional requirements including distributed input processing, parallel arithmetic operations, fault-tolerant communication, synchronous result aggregation, performance monitoring, and scalable container orchestration. The Docker-based deployment demonstrated practical distributed systems concepts while providing comprehensive performance evaluation against centralized alternatives.

10.2 Performance Insights

Experimental results reveal that communication overhead dominates computational benefits for simple arithmetic operations, with the distributed system underperforming centralized processing by 50-140x in throughput and 100-160x in latency. Network round-trip times and MPI message passing costs completely overshadow arithmetic computation time, demonstrating that distributed architectures prove counterproductive when communication overhead exceeds processing complexity. The findings validate that distributed systems excel for computationally intensive workloads but become inefficient for lightweight operations.

10.3 Design Lessons

The project highlighted critical trade-offs between educational objectives and performance optimization in distributed system design. Key lessons include the importance of matching system architecture to workload characteristics, understanding when centralized solutions outperform distributed alternatives, and recognizing that containerized MPI deployments introduce significant but manageable complexity. The implementation successfully demonstrated distributed systems principles while revealing the practical constraints that determine when distributed architectures provide genuine benefits versus educational value.

11 References

- [1] MPI Documentation. Available: <https://MPI.io/>
- [2] Docker Documentation. Available: <https://docs.docker.com/>